

层和块

- 块 (block) 可以描述单个层、由多个层组成的组件或整个模型本身
 - 可以把多个层按顺序打包成整体
- 块由类Class表示
 - 任何子类都必须定义一个将输入转换成输出的钱箱传播函数，且存储必需的参数。

类：相当于一个模板

```
Class Dog:
    def __init__(self, name):
        # __init__是创建对象时自动调用的初始化函数
        self.name = name        # self指对象自己

    def bark(self):              # 定义行为
        print(f"{self.name}: 汪!")

# 用模板造出具体对象
dog1 = Dog("black") # 调用__init__, self.name = "black"
dog1.bark()         # 输出 black: 汪!
```

继承

```
class Animal:
    def eat(self):
        print(eating)

class Dog(Animal): # 继承了Animal

dog = Dog()
dog.eat()          # 可正常使用，输出eating
```

__call__ —— 让对象**像函数一样调用**

```
class Adder
    def __call__(self,x,y): # 定义了__call__就能用()调用
        return x+y

add = Adder()
add(3,5) # 触发了__call__, 返回8(3+5)
```

实例代码

```
net = nn.Sequential(nn.Linear(20,256), # 一个Module
                    nn.ReLU(),         # 也是一个Module
                    nn.Linear(256,10)) # 还是一个Module

X = torch.rand(2,20)
net(X)
```

- nn.Sequential定义了特殊的Module，从Module那继承而来
 - 即在Pytorch上表示一个块的类
 - 维护了由Module组成的有序列表
- 2个全连接层都是Linear类的实例，Linear类本身是Module的子类
- net(X)调用模型获得输出，本质上是net.__call__(X)

前向传播逻辑：将列表中每个块连在一起，每个块输出作为下一个块输入

自定义块

块的基本功能

- 将input作为前向函数的参数
- 通过前向函数生成output
- 计算output关于input的梯度
- 存储和访问前向传播计算所需的参数
- 根据需要来初始化模型参数

示例代码：定义自己的神经网络，与之前的nn.Sequential()不同

```

class MLP(nn.Module): # 定义一个继承于nn.Module的MLP类
    # 用模型参数来声明层，以下声明两个全连接
    def __init__(self): # 初始化函数，创建对象时自动调用，准备需要的零件
        super().__init__() # 调用父类的初始化
        self.hidden = nn.Linear(20,256) # 准备第一个零件，即隐藏层
        self.out = nn.Linear(256,10) # 准备第二个零件，即输出层
    # self指的是MLP这个对象自己，先调用好父类初始化，自己再装2个层

    def forward(self, X):
        return self.out( # 把h'松紧输出层out算一下，得到最终结果
            F.relu( # 对h做激活函数ReLU，的h'
                self.hidden(X) # X进入隐藏层算一下，得到h
            )
        )

net = MLP() #一种实例化

```

之前的简洁写法：

```

net = nn.Sequential(
    nn.Linear(20,256),
    nn.ReLU(),
    nn.Linear(256,10)
)

```

代码大体逻辑

- **前向传播函数**forward(self,x)
 - 以x作为输入
 - 计算带有激活函数F.relu的隐藏hidden表示
 - 输出self.out其为规范化的输出值
- **初始化函数**__init__(self)
 - 通过super().__init__()调用父类的__init__函数，不重复造轮子
 - 实例化2个全连接层，即self.hidden和self.out

顺序块

Sequential

- 目的：把其他模块串起来
- 关键的2个函数

- 将块逐个追加到列表的函数，形成固定顺序
- 前向传播函数，按照顺序传递数据

前向传播中执行代码

当需要更好的灵活性时，我们不能只依靠Sequential，而是需要自定义块，例如，我希望

- 中间复用一层
- forward里加while循环
- 固定一个随机权重，即**常数参数**

示例代码

```
class FixedHiddenMLP(nn.Module):
    def __init__():
        super().__init__()
        # 设置常数参数
        self.rand_weight = torch.rand((20,20), requires_grad = False)
        self.linear = nn.Linear(20,20)

    def forward(self, X):
        X = self.linear(X) # 过标准线性层
        X = F.relu(torch.mm(X, self.randweight) + 1) # 和固定权重做矩阵乘法
        X = self.linear(X) # 复用一层，共享参数
        while X.abs().sum() > 1: #
            X /= 2
        return X.sum()
```

嵌套块的示例代码

```

class NestMLP(nn.Module):
    def __init__(self):
        super().__init__() # 依旧子承父业
        self.net = nn.Sequential( # 定义一个嵌套块
            nn.Linear(20, 64), # 定义嵌套块的第一层，以下同理
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU()
        )
        self.linear = nn.Linear(32,16) # 定义嵌套块外面的全连接层

    def forward(self, X):
        return self.linear( # 再过全连接层
            self.net(X) # 先过嵌套块
        )

chirema = nn.Sequential(
    NestMLP(),
    # 同时完成了：实例化NestMLP + 将实例传进Sequential
    nn.Linear(16,20),
    FixedHiddenMLP()
)

```

参数管理

实例

```

net = nn.Sequential(
    nn.Linear(4,8),
    nn.ReLU(),
    nn.Linear(8,1)
)

X = torch.rand(size=(2,4))

```

参数访问

当通过Sequential类定义模型时，可以通过索引访问任意层，可以把模型比作列表

访问第二个全连接层参数

```
print(net[2].state_dict())
```

访问第二个全连接层的bias

```
print(net[2].bias)
```

参数初始化

nn.init模块提供了多种预置初始化方法

默认随机初始化

```
net = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 1))
```

内置初始化

- 将权重w初始化std为0.01的高斯随机变量，bias设置为0

```
def init_normal(m):  
    if type(m) == nn.Linear:  
        nn.init.normal_(m.weight, mean=0, std=0.01)  
        nn.init.zeros_(m.bias)  
net.apply(init_normal)
```

- 将参数初始化为给定常数，如1

```
def init_normal(m):  
    if type(m) == nn.Linear:  
        nn.init.normal_(m.weight, 1) #这行变了  
        nn.init.zeros_(m.bias)  
net.apply(init_normal)
```

延后初始化

第一次传数据前，参数还没初始化

自定义层

如果 PyTorch 没有你想要的层，可以自己写一个

读写文件

即保存模型，保存中间结果

加载和保存张量

- 单个张量
 - 可以调用load和save来读写
 - 均需要提供一个名称

```
x = torch.arange(4)
torch.save(x, 'x-file')
# 紧接着将存储在文件的数据读回内存
x2 = torch.load('x-file')
```

以下省略，用到的时候再查

加载和保存模型参数

保存的是 **参数** 而不是整个模型

- 因为模型本身包含任意代码，难以序列化
- 为了恢复模型，还需用代码生成架构，从磁盘加载参数

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(20, 256)
        self.output = nn.Linear(256, 10)

    def forward(self, X):
        return self.output(F.relu(self.hidden(X)))

net = MLP()
X = torch.randn(size=(2, 20))
Y = net(X)

# 将模型参数存储在名为'mlp.params'的文件中
torch.save(net.state_dict(), 'mlp.params')
```

恢复模型

```
clone = MLP() # 实例化原始MLP的一个备份
clone.load_state_dict(torch.load('mlp.params')) # 加载模型参数
clone.eval() # 相当于加载好模型参数后，开始搞测试
```

GPU

计算设备

- CPU
 - 用 `torch.device('cpu')` 表示
 - 意味所有物理CPU和内存，Pytorch的计算将尝试使用CPU核心
- GPU
 - 用 `torch.device('cuda')` 表示
 - 只代表一个卡和相应的内存
 - 如果有多张卡，可以用 `torch.device(f'cuda:{i}')` 来表示第i块CPU
 - i从0开始
 - `cuda:0` 和 `cuda`等价

检查有没有GPU

```
import torch
torch.cuda.is_available() # True 表示有，False 表示没有
```

把模型和数据放到 GPU 上跑

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# 把模型放到 GPU
net = MLP().to(device)

# 把数据也放到 GPU
X = torch.rand(2, 20).to(device)

# 然后在 GPU 上计算
Y = net(X)
```